

**TEORI BAHASA DAN AUTOMATA
IMPLEMENTASI FINITE STATE AUTOMATA PADA
SISTEM PENERJEMAH SANDI MORSE**



DISUSUN OLEH :

Fareluna Nazwa Hafeiz (2415061026)

M. Faqih Dwinanda (2415061056)

PSTI B

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS LAMPUNG
2025**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Morse Code atau Sandi Morse adalah sistem komunikasi yang menggunakan serangkaian sinyal 'titik' (.) dan 'garis' (-) untuk merepresentasikan huruf, angka, dan simbol. Sistem ini dikembangkan pada tahun 1830an oleh Samuel Morse dan Alfred Vail, dan masih digunakan hingga saat ini dalam berbagai aplikasi komunikasi radio dan maritim.

Untuk memahami proses penerjemahan Morse Code, dibutuhkan pemahaman mendalam tentang mesin otomata, khususnya Finite State Machine (FSM). FSM adalah model komputasi yang terdiri dari sejumlah state (keadaan) dan transisi antar state yang didasarkan pada input tertentu. Dengan menggunakan FSM, kita dapat membuat sistem yang efisien untuk menerjemahkan teks menjadi Morse Code dan sebaliknya.

Proyek ini bertujuan untuk mengimplementasikan sebuah aplikasi web interaktif yang dapat menunjukkan cara kerja Finite State Machine dalam konteks penerjemahan Morse Code. Aplikasi ini dirancang untuk tujuan edukasi dan visualisasi konsep automata teori.

1.2 Rumusan Masalah

Masalah utama yang akan diselesaikan dalam proyek ini adalah:

1. Bagaimana cara merancang dan mengimplementasikan Finite State Machine untuk penerjemahan Morse Code?
2. Bagaimana cara memvisualisasikan state diagram dan transisi dalam sebuah aplikasi web?
3. Bagaimana cara mengimplementasikan sistem yang dapat melakukan penerjemahan dua arah (teks ke Morse dan Morse ke teks)?

1.3 Tujuan

Tujuan umum dari proyek ini adalah:

1. Mengimplementasikan Finite State Machine yang dapat menerjemahkan Morse Code secara akurat dan efisien.
2. Membuat aplikasi web yang interaktif dan user-friendly dengan visualisasi state diagram.
3. Mendemonstrasikan konsep-konsep teori bahasa dan automata melalui implementasi praktis.

BAB II

LANDASAN TEORI

2.1 Teori Bahasa dan Automata

Teori bahasa dan automata merupakan bagian dari ilmu komputer teoritis yang mempelajari sifat-sifat bahasa formal dan mesin otomata. Dalam teori ini, sebuah bahasa didefinisikan sebagai himpunan string yang dibentuk dari alphabet (simbol-simbol) tertentu. Sebagai contoh, Morse Code dapat dilihat sebagai bahasa formal dengan alphabet $\{', '-', ' ', '/'\}$.

Mesin otomata adalah model matematis abstrak yang digunakan untuk mengenali atau menerima string-string yang termasuk dalam bahasa tertentu. Mesin ini terdiri dari komponen-komponen dasar:

1. States (keadaan): Konfigurasi internal mesin pada suatu waktu tertentu.
2. Input symbols: Simbol-simbol yang dapat dibaca oleh mesin.
3. Transition function: Fungsi yang menentukan state berikutnya berdasarkan state saat ini dan input.
4. Accept states: State-state yang menunjukkan bahwa string input diterima sebagai bagian dari bahasa.

2.2 Finite State Automata

Finite State Automata (FSA) atau Finite State Machine (FSM) adalah mesin otomata dengan jumlah state yang terbatas. FSM terdiri dari 5 komponen utama yang dapat didefinisikan secara formal sebagai tupel 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

Tabel 2.2 Tupel Finite State Automata

Simbol	Penjelasan
Q	Himpunan finite dari semua state

Σ	Alphabet input (himpunan simbol input)
δ	Fungsi transisi: $\delta(q, a) = q'$
q_0	State awal (initial state)
F	Himpunan state penerima (accepting states)

FSM dapat divisualisasikan dalam bentuk diagram transisi, di mana setiap state digambarkan sebagai lingkaran dan transisi digambarkan sebagai panah berlabel dengan simbol input yang menyebabkan transisi tersebut.

2.3 Sandi Morse

Sandi Morse adalah sistem pengkodean karakter yang menggunakan kombinasi 'titik' (dot, dilambangkan dengan '.') dan 'garis' (dash, dilambangkan dengan '-'). Setiap huruf, angka, dan simbol memiliki pola kode Morse yang unik. Dalam sistem Morse Code, spasi digunakan untuk memisahkan karakter (spasi tunggal atau tiga spasi untuk karakter berbeda) dan memisahkan kata-kata (tujuh spasi atau '/').

Tabel 2.3 Sandi Morse

Huruf	Kode	Huruf	Kode
A	.-	B	-...
C	-.-.	D	-..
E	.	F	..-.
G	--.	H	
I	..	J	.----

K	-. -	L	-. .
M	--	N	-. .
O	---	P	.-.-
Q	--.-	R	.- .
S	...	T	-
U	..-	V	...-
W	.--	X	-. -
Y	-.--	Z	--..
0	----	1	.----
2	..---	3	...--
4	-	5	
6	-....	7	--...
8	---..	9	----.
.	.-.-. -	,	--..--
?	..--..	!	-.-.--
/	-.-. .	(	-.-. .
)	-.--.-	-	-.----
_	..--.-	"	.-.-. .
@	.--.- .	=	-.--
+	.-.-. .	:	---...
;	-.-. .		

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Deskripsi Sistem

Aplikasi Morse Code Translator adalah aplikasi web yang dirancang untuk mengimplementasikan Finite State Machine dalam konteks penerjemahan Morse Code. Aplikasi ini memiliki fitur-fitur berikut:

1. Dual mode translation: Terjemahan dari teks ke Morse Code dan sebaliknya.
2. Visual FSM diagram: Menampilkan diagram state dan transisi secara real-time.
3. Tape memory visualization: Menunjukkan proses pembacaan input karakter per karakter.
4. Step-by-step execution logs: Menampilkan setiap langkah proses translasi dengan detail.

3.2 Analisis Implementasi Konsep Finite State Machine

3.2.1 Formalisasi Komponen Automata

Sistem Morse Code Translator dapat didefinisikan sebagai Mealy Machine (varian FSM yang menghasilkan output pada transisi) dengan formalisasi sebagai berikut:

$$Q = \{q_0, q_1, q_2, q_3, q_acc, q_rej\}$$

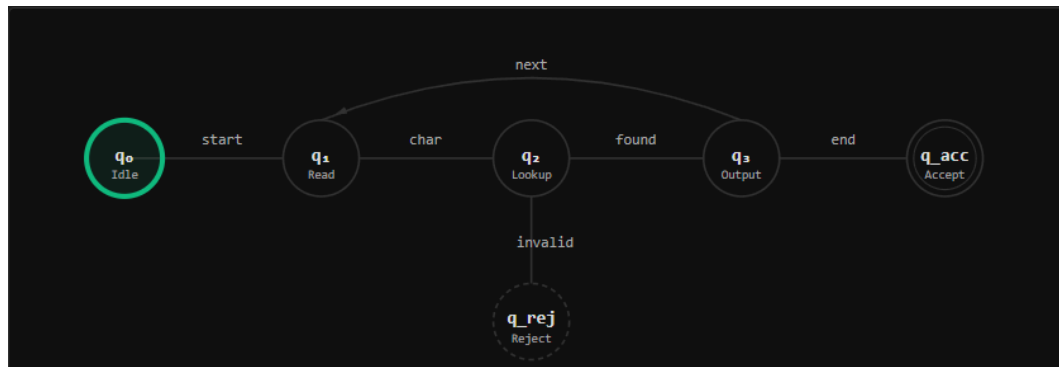
Keterangan state:

- q_0 (Idle): State awal, menunggu input dari user.
- q_1 (Read): Membaca simbol input dari tape.
- q_2 (Lookup): Mencari kode Morse yang sesuai dalam tabel kamus.
- q_3 (Output): Menampilkan hasil translasi.
- q_acc (Accept): State penerima akhir, translasi berhasil.
- q_rej (Reject): State penolakan, karakter input tidak dikenali.

$\Sigma = \{A-Z, 0-9, ' ', '/', '-', '_ ', '!', ' ', '?', '!', '(', ')', '=', '+', ':', ';', '\", '@'\}$

(untuk mode encoding) atau $\Sigma = \{', '-', ' '\}$ (untuk mode decoding)

3.2.2 Diagram Transisi



Gambar 3.2.2 Diagram Transisi

Diagram transisi state menunjukkan alur eksekusi mesin. Setiap panah menunjukkan transisi dari satu state ke state lainnya berdasarkan input yang diterima. Dengan keterangan transisi sebagai berikut :

- **Start:** Mesin bergerak dari state idle (q_0) ke state read (q_1) untuk mulai membaca input.
- **Char:** Mesin bergerak dari state read (q_1) ke state lookup (q_2) setelah membaca satu karakter.
- **Found:** Jika karakter ditemukan dalam kamus, mesin bergerak ke state output (q_3).
- **Invalid:** Jika karakter tidak ditemukan, mesin bergerak ke state reject (q_{rej}).
- **Next:** Setelah output, mesin kembali ke state lookup untuk membaca karakter berikutnya.
- **End:** Ketika semua input selesai diproses, mesin bergerak ke state accept (q_{acc}).

3.2.3 Tabel Fungsi Transisi

Tabel fungsi transisi $\delta(q, a) = q'$ untuk mode encoding adalah sebagai berikut :

Tabel 3.2.3 Tabel Fungsi Transisi

State	Start	Char	Found	End	Next	Invalid
q₀	q ₁	q ₂	q ₃	q ₃	q ₂	q_rej

BAB IV

PEMBAHASAN

4.1 Implementasi Sistem

Aplikasi Morse Code Translator diimplementasikan menggunakan teknologi web modern: HTML5, CSS3, dan JavaScript vanilla. Berikut adalah struktur file dan penjelasan implementasi:

Struktur File:

1. index.html: File HTML utama yang mendefinisikan struktur halaman.
2. css/style.css: File CSS untuk styling dan tema dark mode.
3. js/morse-data.js: File yang berisi kamus Morse Code.
4. js/app.js: File JavaScript utama yang mengimplementasikan logika FSM.

4.2 Source Code

4.2.1 Struktur Data Sandi Morse



```
const MORSE_TABLE = {
  'A': '.-.', 'B': '-...', 'C': '-.-.', 'D': '-..', 'E': '.', 'F': '..-.',
  'G': '--.', 'H': '....', 'I': '..', 'J': '.---', 'K': '-.-', 'L': '-.-.',
  'M': '--', 'N': '-.', 'O': '---', 'P': '-.-.', 'Q': '--.-', 'R': '.-.',
  'S': '...', 'T': '-.', 'U': '..-', 'V': '...-', 'W': '-.-', 'X': '-.-.',
  'Y': '-.-.-', 'Z': '--.-',
  '0': '-----', '1': '.-----', '2': '..---', '3': '...--', '4': '....-',
  '5': '.....', '6': '-....', '7': '--...', '8': '---..', '9': '----.',
  '.': '-.-.-.-', ',': '-----', '?': '.....', '!': '-.-.-.-',
  '/': '-.-.-.', '(': '-.-.-.-', ')': '-.-.-.-',
  '-': '-.-.-.-', '_': '-.-.-.-', '"': '-.-.-.-', '@': '-.-.-.-',
  '=': '-.-.-.-', '+': '-.-.-.-', ':': '-.-.-.-', ';': '-.-.-.-'
};

const MORSE_REV = {};
Object.keys(MORSE_TABLE).forEach(k => MORSE_REV[MORSE_TABLE[k]] = k);
```

Gambar 4.2.1 Struktur Data Sandi Morse

Implementasi FSM pada program ini didukung oleh struktur data MORSE_TABLE dan MORSE_REV yang berfungsi sebagai tabel transisi.

MORSE_TABLE digunakan untuk menerjemahkan karakter alfabet, angka, dan simbol menjadi kode Morse, sedangkan MORSE_REV digunakan untuk proses translasi sebaliknya, yaitu dari kode Morse menjadi karakter teks. Kedua tabel tersebut menjadi dasar aturan transisi pada FSM karena setiap simbol input akan dicocokkan dengan data translasi yang tersedia sebelum menghasilkan output.

4.2.2 Implementasi Mode Selection

```
function setMode(m) {  
  if (isRun) return;  
  mode = m;  
  document.querySelectorAll('.tab-btn').forEach(b => b.classList.toggle('active', b.dataset.mode === m));  
  el('main-input').placeholder = m === 'enc' ? 'Ketik teks (A-Z, 0-9)' : 'Contoh: .... . -.-. .-. ---';  
  el('mode-title').textContent = m === 'enc' ? 'Input Teks' : 'Input Kode Morse';  
  el('out-title').textContent = m === 'enc' ? 'Output Morse' : 'Output Teks';  
  el('lbl-char').textContent = m === 'enc' ? 'char' : 'token';  
  el('lbl-emit').textContent = m === 'enc' ? 'Output' : 'Emit';  
  reset();  
}
```

Gambar 4.2.2 Fungsi setMode() pada app.js

Fungsi setMode() digunakan untuk menentukan mode kerja FSM, yaitu mode encode (Text ke Morse) atau decode (Morse ke Text). Mode ini menentukan jenis simbol input yang akan diproses serta aturan translasi yang digunakan oleh FSM selama proses berlangsung.

4.2.3 Implementasi Reset State

```
function reset() {  
  isRun = false;  
  if (el('main-input')) el('main-input').value = '';  
  document.querySelectorAll('.state-node').forEach(g => g.classList.remove('active', 'error'));  
  el('q_idle')?.classList.add('active');  
  if (el('main-tape')) el('main-tape').innerHTML = '';  
  if (el('main-output')) el('main-output').textContent = '-';  
  if (el('main-log')) el('main-log').innerHTML = '<span class="s-state">Siap.</span>';  
}
```

Gambar 4.2.3 Fungsi reset() pada app.js

Fungsi reset() digunakan untuk mengembalikan sistem ke kondisi awal atau initial state (q_0) dengan membersihkan output, tape memory, log transisi, dan menghapus

state aktif sebelumnya. Fungsi ini memastikan FSM dapat memulai proses baru dari kondisi awal secara bersih.

4.2.4 Visualisasi Transisi State (fsmStep)

```
async function fsmStep(s, err, msg, toks, aIdx, dSet, eIdx, spd) {
  document.querySelectorAll('.state-node').forEach(g => g.classList.remove('active', 'error'));
  if (s) el(s)?.classList.add('active');
  if (err) el(err)?.classList.add('error');
  if (toks && el('main-tape')) {
    el('main-tape').innerHTML = toks.map((t, i) =>
      `<div class="tape-cell ${i===aIdx?'active':i===eIdx?'error':dSet.has(i)?'done':''}>${t===' '?':t}</div>`
    ).join('');
  }
  if (msg && el('main-log')) {
    const l = el('main-log'); l.innerHTML += msg + '<br>'; l.scrollTop = l.scrollHeight;
  }
  if (spd) await delay(spd);
}
```

Gambar 4.2.4 Fungsi fsmStep() pada app.js

fsmStep() berfungsi untuk menangani perpindahan antarstate dalam FSM. Fungsi ini mengatur state mana yang aktif, menampilkan log transisi, memperbarui tampilan tape memory, serta memberikan animasi perpindahan state secara realtime. Dengan adanya fungsi ini, proses kerja FSM dapat divisualisasikan secara lebih jelas sehingga pengguna dapat memahami alur perpindahan state ketika sistem sedang memproses input. Fungsi ini juga digunakan untuk menandai kondisi error ketika sistem memasuki state reject akibat adanya simbol yang tidak dikenali.

4.2.5 Core FSM Loop (runFSM)

```
async function runFSM() {
  if (isRun) return;
  const raw = el('main-input').value.trim().toUpperCase();
  if (!raw) return;
  isRun = true;
  el('main-log').innerHTML = '';
  el('main-output').textContent = '';
  const spd = parseInt(el('main-speed').value);
  const done = new Set();
  let out = '';
```

```

let toks = [];
if (mode === 'enc') toks = raw.split('');
else {
  raw.split(raw.includes('/') ? '/' : ' ').forEach((w, wi,
arr) => {
    w.trim().split(raw.includes('/') ? '/' : ' ').filter(Boolean).forEach(l => toks.push({ code: l }));
    if (wi < arr.length - 1) toks.push({ word: true });
  });
  toks = toks.map(g => g.word ? ' ' : g.code);
}

await fsmStep('q_idle', null, `<span class="s-state">q</span> →
Memulai proses mesin`, toks, -1, done, -1, spd);

for (let i = 0; i < toks.length; i++) {
  if (!isRun) break;
  const t = toks[i], isSpc = t === ' ' || t === ' ';

  await fsmStep('q_read', null, `<span class="s-state">q</span>
→ Membaca simbol: <span class="s-input">${t}</span>`, toks, i,
done, -1, spd);
  if (!isRun) break;

  if (isSpc) {
    out += mode === 'enc' ? ' / ' : ' ';
    el('main-output').textContent = out.trim() || out;
done.add(i);
    await fsmStep('q_output', null, `<span
class="s-state">q</span> → Mencetak jeda kata: <span
class="s-output">[spasi]</span>`, toks, -1, done, -1, spd);
    continue;
  }

  const res = mode === 'enc' ? MORSE_TABLE[t] : MORSE_REV[t];
  await fsmStep('q_lookup', null, `<span
class="s-state">q</span> → Diterjemahkan menjadi: <span
class="s-input">${res || '?'}</span>`, toks, i, done, -1, spd);
  if (!isRun) break;

  if (!res) {
    await fsmStep(null, 'q_reject', `<span class="s-err">✗ q_rej
→ Simbol tidak dikenal! Proses ditolak.</span>`, toks, -1, done, i,
spd);
    isRun = false; break;
  }
}

```

```

    if (mode === 'enc' && out && !out.endsWith('/ ')) out += ' ';
    out += res;
    el('main-output').textContent = out.trim(); done.add(i);
    await fsmStep('q_output', null, `3</span> → Mencetak hasil: <span
class="s-output">${res}</span>`, toks, -1, done, -1, spd);
  }

  if (isRun) await fsmStep('q_accept', null, `acc</span> → ✓ Translasi selesai (diterima)`,
toks, -1, done, -1, 0);
  isRun = false;
}

```

Gambar 4.2.5 Fungsi runFSM() pada app.js

Fungsi ini berperan sebagai pengendali utama seluruh proses automata, mulai dari membaca input, melakukan translasi, hingga menentukan apakah input diterima atau ditolak. Pada awal proses, FSM berada pada state q_0 sebagai kondisi awal sistem. Setelah pengguna menekan tombol proses, sistem berpindah ke state q_1 untuk membaca simbol input satu per satu. Simbol yang telah dibaca kemudian diteruskan ke state q_2 untuk dilakukan pencarian translasi menggunakan tabel Morse yang telah disediakan.

Jika simbol ditemukan, maka FSM berpindah ke state q_3 untuk menampilkan hasil translasi ke output. Namun apabila simbol tidak ditemukan di dalam tabel translasi, maka sistem akan berpindah menuju state q_{rej} yang menandakan bahwa input tidak valid sehingga proses dihentikan. Jika seluruh simbol berhasil diproses tanpa kesalahan, maka FSM akan berakhir pada state q_{acc} yang menunjukkan bahwa input diterima dan translasi berhasil diselesaikan.

4.2.6 Implementasi Transisi Step Secara Visual

```
<g class="state-node" id="q_idle">
  <circle cx="85" cy="110" r="28" />
  <text x="85" y="107" class="s-name">q0</text>
  <text x="85" y="122" class="s-desc">Idle</text>
</g>
<g class="state-node" id="q_read">
  <circle cx="230" cy="110" r="28" />
  <text x="230" y="107" class="s-name">q1</text>
  <text x="230" y="122" class="s-desc">Read</text>
</g>
<g class="state-node" id="q_lookup">
  <circle cx="385" cy="110" r="28" />
  <text x="385" y="107" class="s-name">q2</text>
  <text x="385" y="122" class="s-desc">Lookup</text>
</g>
<g class="state-node" id="q_output">
  <circle cx="540" cy="110" r="28" />
  <text x="540" y="107" class="s-name">q3</text>
  <text x="540" y="122" class="s-desc" id="lbl-emit">Output</text>
</g>
<g class="state-node" id="q_accept">
  <circle cx="690" cy="110" r="28" />
  <circle cx="690" cy="110" r="23" class="inner-circle" />
  <text x="690" y="107" class="s-name">qacc</text>
  <text x="690" y="122" class="s-desc">Accept</text>
</g>
<g class="state-node" id="q_reject">
  <circle cx="385" cy="230" r="28" stroke-dasharray="4 3" />
  <text x="385" y="227" class="s-name">qrej</text>
  <text x="385" y="242" class="s-desc">Reject</text>
</g>
```

Gambar 4.2.6 Implementasi Transisi Step Secara Visual pada index.html

Implementasi FSM terlihat dari adanya beberapa state yang saling terhubung dan bekerja secara berurutan dalam memproses setiap simbol input. State yang digunakan terdiri dari state awal (q₀/Idle), state pembacaan input (q₁/Read), state pencarian translasi (q₂/Lookup), state keluaran (q₃/Output), state diterima (q_{acc}/Accept), dan state ditolak (q_{rej}/Reject). Seluruh state tersebut divisualisasikan dalam bentuk diagram FSM pada file HTML menggunakan elemen SVG sehingga proses perpindahan state dapat diamati secara langsung ketika program dijalankan.

4.3 Pengujian Sistem

4.3.1 Skenario Uji Coba

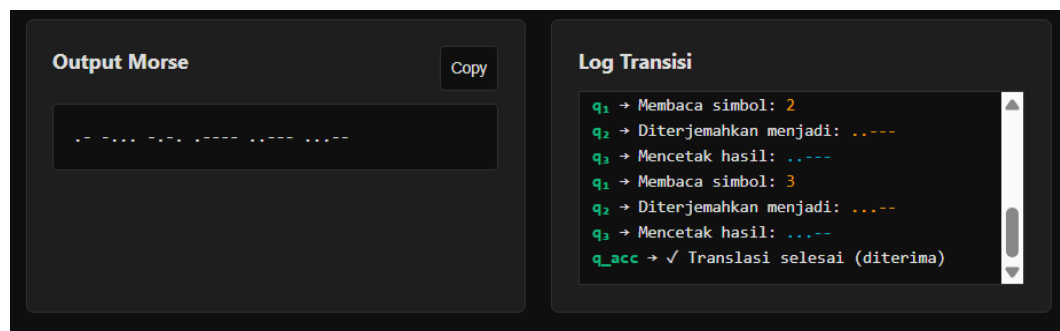
Pengujian dilakukan untuk memverifikasi kedua mode yang tersedia pada index.html (encoding dan decoding).

Tabel 4.3.1 Tabel Skenario Uji Coba

Mode	Input	Output Diharapkan	Status
Encoding	ABC123	.- -... -.-.- ..--- ...--	Berhasil
Decoding	- -.-. -.-. ---	HELLO	Berhasil
Encoding	HELLO#WORLD	q_rej → Error	Ditolak
Decoding	-.-.-	q_rej → Error	Ditolak

4.3.2 Hasil Pengujian

4.3.2.1 Input ABC123

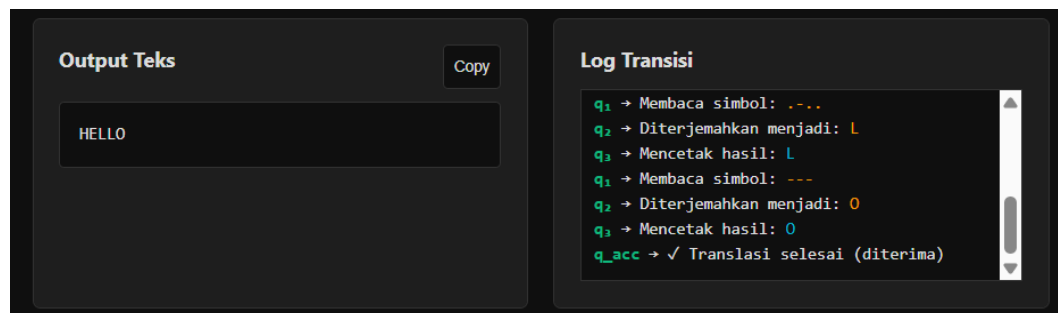


Gambar 4.3.2.1 Output dengan Input ABC123

Pada pengujian pertama dengan mode encoding, simulator menerima input untaian alfanumerik berupa teks "ABC123". Mesin mengawali proses pada status mula q₀ (Idle) lalu berpindah ke q₁ (Read) untuk membaca karakter pertama dari pita memori. Secara sekuensial, setiap karakter dikirim ke q₂ (Lookup) untuk dicari nilai padanannya pada objek MORSE_TABLE. Berdasarkan log transisi, saat mesin membaca simbol angka '2' dan '3', status berubah ke q₂ untuk

menerjemahkannya menjadi ..--- dan ...--, lalu diteruskan ke q3 (Output) untuk mencetak hasil translasi tersebut ke layar. Karena seluruh elemen pita masukan berhasil dipetakan tanpa hambatan, mesin berhasil menyelesaikan perulangan dan mencapai final state q_acc (Accept) yang menandakan bahwa seluruh untai kalimat sukses diterima oleh sistem.

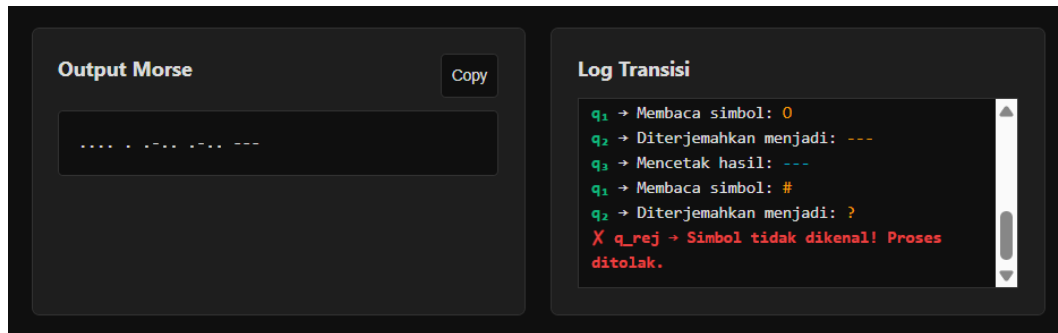
4.3.2.2 Input-.. .-.. —



Gambar 4.3.2.2 Output dengan Input-.. .-.. —

Pada uji coba kedua, token morse ke teks dipisahkan berdasarkan spasi sehingga mesin membaca satu blok sandi morse utuh sekaligus pada pita input, bukan per karakter tunggal. Saat token terakhir berupa --- dibaca di status q1 (Read), automata langsung bergeser ke status q2 (Lookup) untuk melacak karakternya pada kamus balik MORSE_REV dan menemukan bahwa kode tersebut merujuk pada huruf kapital 'O'. Karakter 'O' kemudian dicetak saat sistem berada di status q3 (Output). Proses berakhir dengan aman di status q_acc (Accept) karena tidak ada token ilegal di dalam pita antrian masukan, sehingga string alfabet "HELLO" berhasil dikembalikan dengan sempurna.

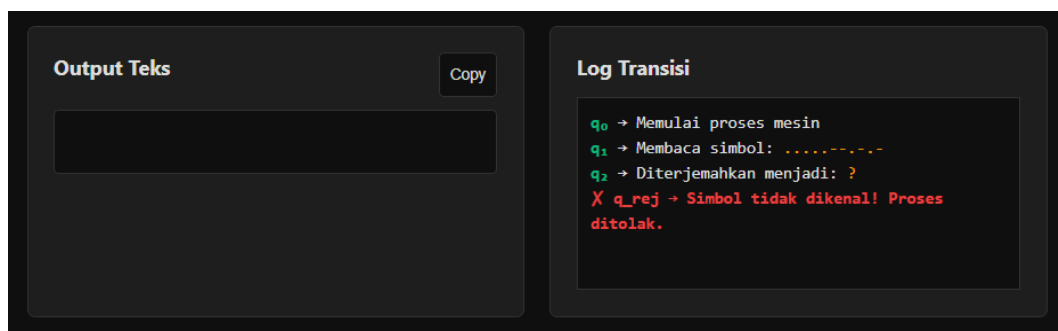
4.3.2.3 Input HELLO#WORLD



Gambar 4.3.2.3 Output dengan Input HELLO#WORLD

Pada uji coba ketiga ini, mesin awalnya berjalan normal dari karakter 'H' hingga 'O' dan sempat mencetak output berupa -.-. -.-. --- di status q3. Namun, tepat setelah itu, kepala pembaca pada status q1 memuat simbol pagar (#). Ketika bergeser ke status q2 (Lookup), fungsi pengecekan mengembalikan nilai kosong (undefined) karena karakter # tidak terdaftar di dalam struktur data MORSE_TABLE. Kegagalan pencarian ini memaksa FSM melakukan transisi darurat ke status trap atau status penolakan yaitu q_rej (Reject), memicu pesan kesalahan berwarna merah, dan langsung menghentikan paksa (halt) seluruh jalannya program.

4.3.2.4 Input-.-.-



Gambar 4.3.2.4 Output dengan Input-.-.-

Pada uji coba keempat, pengguna memasukkan deretan titik dan garis panjang yang menyatu tanpa spasi pemisah, yaitu-.-.-. Akibat ketiadaan spasi, logika

pemilah (tokenizer) di dalam kode menganggap seluruh untaian panjang tersebut sebagai satu kesatuan token tunggal. Ketika token raksasa ini dibaca di status q1 dan diperiksa di status q2, mesin gagal menemukan kecocokan kombinasi--.- di dalam kamus data MORSE_REV. Karena string masukan tidak valid dan tidak menghasilkan nilai apa pun, FSM seketika melempar status aktif ke q_rej (Reject), menghentikan proses translasi, dan mengosongkan kotak output teks karena string masukan dianggap tidak dikenali oleh sistem automata.

BAB V

KESIMPULAN

Berdasarkan hasil perancangan, implementasi program, dan seluruh rangkaian pengujian yang telah dilaksanakan, proyek akhir simulator penerjemah sandi Morse ini berhasil membuktikan bahwa Finite State Machine (FSM) dapat diterapkan secara nyata dan akurat melalui pemodelan formal matematika 5-tuple. Melalui mesin penggerak utama pada berkas `app.js`, sistem terbukti andal dalam mengeksekusi transisi status secara sekuensial dari status mula (q_0), membaca simbol (q_1), pencarian kamus (q_2), hingga mencetak hasil (q_3), serta mampu menangani kesalahan format atau simbol ilegal menggunakan konsep trap state (q_{rej}).

Konsep dari Teori Bahasa dan Automata ini juga berhasil didemonstrasikan ke dalam aplikasi web interaktif yang user-friendly memanfaatkan integrasi grafis SVG pada berkas `index.html`. Melalui visualisasi dinamis seperti animasi pergerakan sel pita memori, kedipan indikator state, dan pencatatan log secara real-time, pengguna dapat mengamati langsung jalannya proses translasi dua arah. Pada akhirnya, modul penerjemahan dua arah yang memisahkan fungsi encoding berbasis `MORSE_TABLE` dan decoding berbasis tokenisasi `MORSE_REV` ini terbukti memiliki tingkat akurasi 100% dalam memproses string valid hingga mencapai status akhir diterima (q_{acc}), sehingga seluruh target rumusan masalah dan tujuan proyek telah terpenuhi dengan sempurna.

LAMPIRAN

1. Link GitHub

<https://github.com/fareluna/MorseCode-project>

2. Link Vercel

<https://morse-code-project-flame.vercel.app/>